Universidad de La Habana
Facultad de Matemática y Computación

Diseño e implementación de una plataforma informática para el autorreporte y la gestión operativa de eventos adversos post-vacunación en el Instituto Finlay de Vacunas

Autor:
Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos
MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

28 de abril de 2026

github.com/JabelResendiz/thesis

Dedicación

Agradecimientos

Agradecimientos

Opinión del tutor

Opiniones de los tutores

Resumen

Resumen en español

Abstract

Resumen en inglés

Índice general

1. Estado del Arte **1**
1.1. Análisis de sistemas de autorreporte clínico existentes 1
1.1.1. Vaccine Adverse Event Reporting System (VAERS) 2
1.1.2. Sistema NotificaRAM de la Agencia Española de Medicamentos y Productos Sanitarios 2
1.1.3. Sistema V-safe del Centers for Disease Control and Prevention 3
1.1.4. Análisis comparativo de los sistemas estudiados 4
1.2. Análisis de tecnologías para la implementación del sistema propuesto 6
1.2.1. Criterios técnicos para la selección de tecnologías 7
1.2.2. Tecnologías para el desarrollo del backend 7
1.2.3. Conclusiones sobre las plataformas de desarrollo expuestas . . 9
1.2.4. Tecnologías para el desarrollo del frontend 14
1.2.5. Conclusiones sobre los frameworks de frontend expuestos . . . 17
1.2.6. Sistemas de gestión de bases de datos 18
1.2.7. Conclusiones sobre los sistemas de gestión de bases de datos expuestos 19
1.2.8. Síntesis comparativa de las tecnologías analizadas 20
1.3. Tecnologías y enfoques para la gestión de datos clínicos 20
1.3.1. Estándares de interoperabilidad clínica 21
1.3.2. Modelos de captura de datos clínicos 23
1.3.3. Herramientas para la validación reactiva de formularios 24
1.3.4. Calidad e integración de datos clínicos 24
1.4. Conclusiones del capítulo 25

2. Propuesta **27**

3. Detalles de Implementación y Experimentos **28**

Conclusiones **29**

Recomendaciones **30**

A. Formularios de Referencia **31**
A.1. Formulario VAERS - Octubre 2025 31

Bibliografía **36**

Índice de figuras

1.1. Logotipo del sistema VAERS (vaers.hhs.gov) 2
1.2. Logotipo del sistema NotificaRAM (notificaram.es) 2
1.3. Logotipo del sistema V-safe (vsafe.cdc.gov) 3
1.4. Ejemplo del flujo de interacción del sistema V-safe mediante encuestas digitales 4
1.5. Logo de .NET. 8
1.6. Logo de Spring Boot 8
1.7. Logo de NestJS 9
1.8. Throughput (respuestas por segundo) en .NET, Spring Boot y Node.js (adaptado de TechEmpower Framework Benchmarks) 11
1.9. Prueba de estrés sobre operaciones POST: uso de memoria y CPU (adaptado de *Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot* [18]) 12
1.10. Logo de React 15
1.11. Funcionamiento del Virtual DOM en React (adaptado de *Virtual DOM in React JS*, GeeksforGeeks [24]) 15
1.12. Logo de Angular 16
1.13. Logo de Vue.js 16
1.14. Ejemplos de sistemas de gestión de bases de datos relacionales y NoSQL 18
1.15. Estructura jerárquica del sistema de codificación MedDRA. 22

A.1. Formulario VAERS - Octubre 2025 32

Ejemplos de código

Capítulo 1

Estado del Arte

El presente capítulo tiene como objetivo analizar los fundamentos teóricos, tecnológicos y los sistemas existentes relacionados con la farmacovigilancia digital y el desarrollo de plataformas de autorreporte clínico. Se aborda el estudio de soluciones implementadas a nivel internacional, así como las tecnologías y estándares utilizados en sistemas de salud digital, con el propósito de establecer una base sólida que sustente el diseño e implementación del sistema propuesto. Asimismo, se revisan conceptos relacionados con el modelado de datos clínicos y estándares de interoperabilidad que garantizan la consistencia y calidad de la información.

1.1. Análisis de sistemas de autorreporte clínico existentes

En la farmacovigilancia moderna, los sistemas de autorreporte clínico permiten registrar eventos adversos posteriores a la vacunación de forma estructurada. Estos sistemas se basan principalmente en modelos de vigilancia pasiva, en los cuales la notificación de eventos depende de la iniciativa de los usuarios o profesionales de salud, sin una recolección activa sistemática de datos por parte de las instituciones.

Estas herramientas operan bajo dos modelos principales: la vigilancia pasiva y la vigilancia activa. El modelo pasivo depende exclusivamente de la iniciativa voluntaria de ciudadanos o profesionales para notificar un evento. Por el contrario, el modelo activo o proactivo establece un contacto directo con el usuario mediante encuestas digitales para recopilar datos de salud en tiempo casi real.

Este apartado examina tres sistemas representativos a nivel internacional: VAERS, NotificaRAM y V-safe. El análisis se centra en sus características funcionales, actores involucrados, fortalezas y limitaciones, con el propósito de identificar brechas que orienten el diseño del sistema propuesto.

1.1.1. Vaccine Adverse Event Reporting System (VAERS)

Figura 1.1: Logotipo del sistema VAERS (vaers.hhs.gov)

VAERS es el sistema nacional de farmacovigilancia en los Estados Unidos, administrado por la *Food and Drug Administration* (FDA) y los *Centers for Disease Control and Prevention* (CDC) [1]. Su objetivo principal es detectar señales de seguridad, definidas como patrones inusuales o un volumen elevado de reportes que no fueron identificados durante los ensayos clínicos [2].

El sistema permite la notificación por parte de profesionales de la salud, fabricantes y ciudadanos a través de formularios electrónicos disponibles en su plataforma web, así como mediante formatos descargables en PDF (véase Anexo A.1) [3].

Para garantizar la integridad del repositorio, la legislación estadounidense contempla sanciones civiles y penales ante el suministro intencional de información falsa en los reportes [3]. Esto contribuye a reducir la introducción de datos malintencionados en un sistema de acceso público.

Los reportes alimentan la base de datos CDC WONDER, garantizando la transparencia mediante el acceso público a datos anonimizados [3]. Ante una señal de alerta, el monitoreo escala hacia sistemas de vigilancia activa y evaluación clínica como el *Vaccine Safety Datalink* (VSD) y el proyecto *Clinical Immunization Safety Assessment* (CISA) para confirmar posibles riesgos asociados a la vacunación [2].

1.1.2. Sistema NotificaRAM de la Agencia Española de Medicamentos y Productos Sanitarios

Figura 1.2: Logotipo del sistema NotificaRAM (notificaram.es)

NotificaRAM es la plataforma oficial de farmacovigilancia en España para la notificación de sospechas de reacciones adversas a medicamentos [4]. La Agencia Española de Medicamentos y Productos Sanitarios (AEMPS) gestiona este portal en el marco del Sistema Español de Farmacovigilancia de Medicamentos de Uso Humano (SEFV-H) [5].

El sistema funciona de forma descentralizada, remitiendo los reportes a los Centros Autonómicos de Farmacovigilancia para su validación técnica inicial [4].

Tras esta evaluación, la base de datos FEDRA(Farmacovigilancia Española, Datos de Reacciones Adversas) centraliza todos los registros nacionales [6].

Estos registros alimentan la plataforma europea EudraVigilance, la cual utilizan las agencias reguladoras de la Unión Europea para la detección de señales de seguridad. Finalmente, la red global VigiBase de la OMS incorpora estos hallazgos para el monitoreo internacional [7].

1.1.3. Sistema V-safe del Centers for Disease Control and Prevention

Figura 1.3: Logotipo del sistema V-safe (vsafe.cdc.gov)

Los Centers for Disease Control and Prevention (CDC) diseñaron V-safe como una herramienta digital para monitorear la seguridad vacunal en los Estados Unidos. Tras su lanzamiento inicial en diciembre de 2020 para supervisar las vacunas contra la COVID-19, el sistema extendió su alcance a otras inmunizaciones, incluyendo aquellas contra el virus respiratorio sincitial (VRS) [8].

A diferencia de los modelos de notificación espontánea, V-safe emplea un enfoque de vigilancia activa mediante el contacto proactivo con los usuarios. El sistema envía encuestas digitales por mensajes de texto o correo electrónico para recopilar información directamente del paciente [9].

El seguimiento se realiza mediante encuestas diarias durante la primera semana posterior a la vacunación y posteriormente de forma semanal durante seis semanas adicionales [8, 10].

La Figura 1.4 ilustra el flujo general de interacción del sistema, basado en el envío periódico de encuestas y la recopilación estructurada de datos reportados por los

usuarios [10].

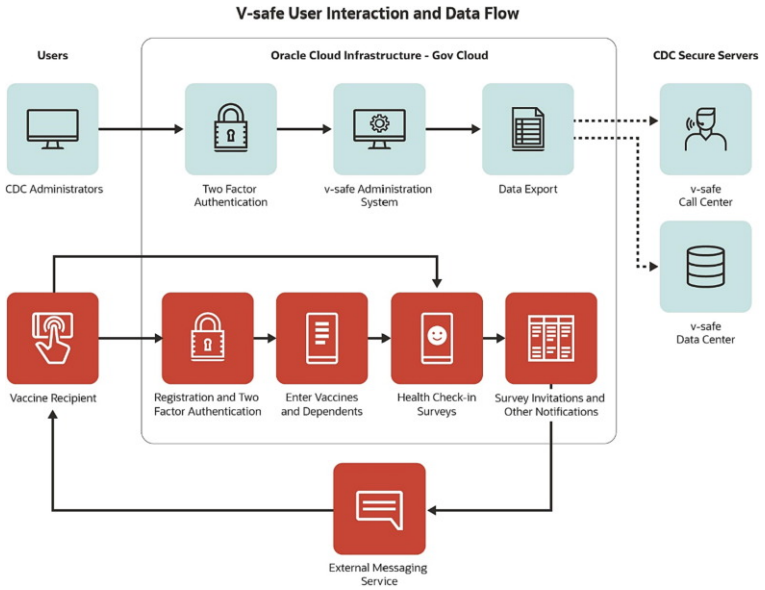

Figura 1.4: Ejemplo del flujo de interacción del sistema V-safe mediante encuestas digitales

V-safe se integra con sistemas de vigilancia pasiva: cuando un usuario reporta la necesidad de atención médica, el sistema facilita la generación de un reporte en VAERS mediante la precarga de datos [9]. Adicionalmente, los datos recopilados se publican en formato anonimizado para su consulta en portales de acceso público.

1.1.4. Análisis comparativo de los sistemas estudiados

Este análisis compara los sistemas VAERS, V-safe y NotificaRAM en términos de su enfoque metodológico, grado de digitalización y mecanismos de captura de datos. La Tabla 1.1 sintetiza estos criterios con el objetivo de evidenciar diferencias estructurales y brechas en los modelos actuales de farmacovigilancia.

Este análisis evidencia diferencias entre los modelos de notificación espontánea y las estrategias de vigilancia activa centradas en el usuario. Mientras que VAERS y NotificaRAM permiten detectar señales mediante la participación voluntaria, V-safe demuestra la escalabilidad de la vigilancia activa al recolectar 151 millones de encuestas de 10 millones de usuarios registrados [10]. Este enfoque, junto con la aplicación de marcos legales de protección de datos, contribuye a la calidad y utilidad científica de la información recolectada.

La interoperabilidad entre plataformas permite la transferencia automatizada de información, como la generación de reportes desde V-safe hacia VAERS y la integra-

Tabla 1.1: Comparación de sistemas de autorreporte de eventos adversos

Criterio	**VAERS**	**NotificaRAM**	**V-safe**
Tipo de vigilancia	Pasiva (Espontánea)	Pasiva (Espontánea)	Activa (Proactiva)
Actores	Profesionales de la salud, fabricantes, ciudadanos	Profesionales de la salud y ciudadanos	Usuarios vacunados registrados
Canal de notificación	Formularios web estructurados y archivos PDF descargables para envío manual	Formulario electrónico único gestionado por la AEMPS para todo el país	Encuestas digitales (SMS y correo electrónico con chequeo de menos de dos minutos)
Protección de datos	Datos anonimizados disponible a través de CDC WONDER; control federal de acceso a información	Datos gestionados bajo marco europeo de protección de datos [6]	Datos desidentificados bajo políticas de protección de datos del sistema federal de salud pública
Estructuración del dato	Baja estructuración; reportes heterogéneos y en gran parte narrativos	Estructuración intermedia mediante formularios estandarizados	Alta estructuración basada en encuestas digitales predefinidas
Fortalezas	Alta cobertura poblacional; acceso público a los datos; detección temprana de señales	Integración en red nacional de farmacovigilancia; análisis coordinado; interoperabilidad con sistemas europeos	Recolección sistemática y periódica; datos en tiempo casi real; seguimiento longitudinal
Limitaciones	Subregistro; sesgos de notificación; calidad variable de los datos; no permite establecer causalidad directa	Subregistro; dependencia de la iniciativa del notificador; variabilidad en la calidad de los datos	Dependencia de acceso a tecnología; participación voluntaria; posible fatiga del usuario; limitada representatividad

ción de FEDRA con EudraVigilance. Esto reduce la latencia y facilita el análisis de señales de seguridad.

A pesar de los avances, los sistemas actuales presentan limitaciones estructurales relevantes, como el subregistro en modelos pasivos, la fatiga del participante en esquemas activos y la imposibilidad de establecer relaciones causales directas sin estudios clínicos complementarios. Asimismo, la dependencia del acceso a tecnologías digitales introduce sesgos en la representatividad de los datos, especialmente en poblaciones con menor conectividad.

Para el desarrollo de plataformas de autorreporte, resultan clave los siguientes aspectos:

- **Interoperabilidad:** Garantizar la integración con sistemas nacionales e internacionales de farmacovigilancia.
- **Protección de datos:** Implementar mecanismos de anonimización o desidentificación que aseguren la confidencialidad de la información, en cumplimiento con los marcos regulatorios de protección de datos vigentes en cada país o región (por ejemplo, normativas nacionales aplicables en el contexto cubano).
- **Calidad del dato:** Implementar esquemas de captura estructurada que garanticen la consistencia del formato de los datos y minimicen la pérdida de información relevante en los reportes.
- **Interacción centrada en el usuario:** Diseñar flujos de contacto proactivo (SMS o correo electrónico) que requieran menos de dos minutos, facilitando el seguimiento longitudinal sin generar fatiga en el usuario.

1.2. Análisis de tecnologías para la implementación del sistema propuesto

El desarrollo de sistemas informáticos modernos orientados al dominio clínico requiere la selección de un conjunto de tecnologías que garanticen propiedades fundamentales como la confiabilidad, la integridad de los datos, la escalabilidad y la mantenibilidad del software. A diferencia de aplicaciones convencionales, en este tipo de sistemas resulta especialmente relevante la correcta gestión de la información, así como la capacidad de respuesta ante un crecimiento progresivo de usuarios y volumen de datos.

En este sentido, el presente análisis se basa en el estudio de diferentes alternativas tecnológicas utilizadas en la construcción de sistemas modernos, abarcando tanto el desarrollo del backend como de las interfaces de usuario y los sistemas de gestión de

bases de datos. La selección de estas tecnologías no se realiza de manera arbitraria, sino a partir de criterios técnicos previamente definidos que permiten evaluar de forma objetiva las distintas opciones disponibles.

1.2.1. Criterios técnicos para la selección de tecnologías

Con el objetivo de realizar una selección fundamentada de las tecnologías que conformarán la solución propuesta, se definen criterios de evaluación orientados a las necesidades de sistemas clínicos.

En primer lugar, se considera la **eficiencia y el rendimiento**, entendidos como la capacidad de la tecnología para responder adecuadamente bajo cargas de trabajo variables, garantizando tiempos de respuesta adecuados en escenarios de uso intensivo.

En segundo lugar, se evalúa la **escalabilidad**, referida a la capacidad del sistema para adaptarse al crecimiento en el número de usuarios y volumen de datos, tanto a nivel vertical como horizontal.

Asimismo, se analiza la **mantenibilidad**, entendida como la facilidad para realizar modificaciones, extensiones o correcciones en el software sin afectar negativamente su funcionamiento global, en función del nivel de acoplamiento y la estructura arquitectónica.

Otro criterio considerado es la **madurez tecnológica**, entendida como el nivel de estabilidad y uso comprobado de una tecnología, reflejado en su adopción en entornos reales, la calidad de su documentación y el soporte de su comunidad.

Finalmente, se introduce el criterio de **adecuación al dominio clínico**, el cual evalúa la capacidad de la tecnología para garantizar propiedades críticas como la consistencia de los datos, la trazabilidad de la información y el soporte de operaciones transaccionales seguras.

Estos criterios sirven como base para el análisis comparativo presentado en las secciones siguientes.

1.2.2. Tecnologías para el desarrollo del backend

En el desarrollo de aplicaciones web modernas, la selección del framework de backend constituye un elemento clave para la construcción del sistema. En el contexto del sistema de farmacovigilancia, se seleccionan tres tecnologías representativas del ecosistema actual: .NET, Spring Boot y NestJS. Esta selección se basa en su amplia adopción en entornos de desarrollo, su uso en aplicaciones de nivel empresarial y la diversidad de paradigmas tecnológicos que representan.

Figura 1.5: Logo de .NET.

.NET Core

.NET (anteriormente .NET Core) es un framework multiplataforma de código abierto mantenido por Microsoft, orientado al desarrollo de aplicaciones backend modernas de alto rendimiento [11].

Entre sus principales características se encuentra el soporte para la construcción de APIs REST [1] mediante ASP.NET Core. Una particularidad de ASP.NET Core es su pipeline de solicitudes HTTP, en el cual cada petición pasa por una secuencia de componentes intermedios (middlewares) que permiten procesarla, validarla o modificarla antes de generar la respuesta [12].

A diferencia de otros ecosistemas, incorpora de manera nativa un contenedor de inyección de dependencias (IServiceProvider), lo que permite que el propio framework gestione el ciclo de vida y la provisión de los componentes sin recurrir a bibliotecas de terceros [13]. Para la persistencia, utiliza Entity Framework Core, un mapeador objeto-relacional (ORM)[2] que permite trabajar con bases de datos relacionales mediante objetos del dominio, simplificando el acceso a los datos [14].

Spring Boot

Figura 1.6: Logo de Spring Boot

[1] API REST es una interfaz HTTP que permite operar recursos entre cliente y el servidor mediante URI y métodos estándar como GET, POST, PUT y DELETE

[2] ORM (Object-Relational Mapping) es una técnica que permite interactuar con bases de datos relacionales mediante objetos del lenguaje de programación, evitando el uso directo de consultas SQL.

Spring Boot es un framework basado en Java, de código abierto, orientado a la construcción de aplicaciones backend mediante un enfoque de configuración automática y convenciones predefinidas [15].

Entre sus principales características se encuentra el uso del paradigma de Inversión de Control (IoC), mediante el cual el framework se encarga de la creación y gestión de los componentes de la aplicación, reduciendo el acoplamiento entre ellos. Asimismo, proporciona soporte para el desarrollo de servicios web mediante Spring MVC, facilitando la implementación de APIs REST con una estructura organizada [16]. Adicionalmente, Spring Boot puede integrarse con Spring Data JPA, lo que permite la interacción con bases de datos relacionales mediante abstracciones orientadas a objetos similares a otros ORM del ecosistema.

NestJS

Figura 1.7: Logo de NestJS

NestJS es un framework de desarrollo backend basado en Node.js y TypeScript, de código abierto, utilizado para la construcción de aplicaciones del lado del servidor y APIs REST [17].

Su arquitectura se organiza mediante módulos, los cuales agrupan controladores y servicios que gestionan la lógica de la aplicación. Utiliza TypeScript como lenguaje base, lo que permite tipado estático dentro del entorno de JavaScript. Se ejecuta sobre el modelo de Node.js basado en un bucle de eventos (event loop), lo que facilita el manejo concurrente de múltiples solicitudes mediante operaciones no bloqueantes. Además, incorpora un sistema de inyección de dependencias y puede integrarse con herramientas como TypeORM o Prisma para el acceso a bases de datos relacionales.

1.2.3. Conclusiones sobre las plataformas de desarrollo expuestas

A partir de las características técnicas descritas previamente, se realiza una comparación de las tecnologías backend seleccionadas (.NET, Spring Boot y NestJS) con base en los criterios definidos en la Sección 1.2.1.

La Tabla 1.2 resume la comparación realizada según los criterios establecidos.

Tabla 1.2: Comparación de tecnologías backend según criterios de evaluación

Criterio	**.NET**	**Spring Boot**	**NestJS**
Eficiencia	Alta (alto throughput, baja latencia, menor consumo de RAM)	Alta (rendimiento estable, mayor consumo de memoria JVM)	Media (eficiente en I/O, menor rendimiento en CPU intensivo)
Escalabilidad	Alta (pipeline ligero, alto rendimiento bajo carga y despliegue eficiente en contenedores)	Muy alta (ecosistema maduro para arquitecturas distribuidas y estabilidad en alta concurrencia)	Alta (alta concurrencia en I/O mediante modelo no bloqueante, modelo asíncrono basado en event loop)
Mantenibilidad	Alta (DI nativo, arquitectura estructurada y tipada)	Alta (IoC consolidado, ecosistema empresarial estandarizado)	Media (modularidad flexible, ecosistema heterogéneo)
Madurez	Muy alta (plataforma enterprise, soporte LTS)	Muy alta (amplia adopción empresarial consolidada)	Media (ecosistema en crecimiento)
Adecuación clínica	Alta (EF Core, transacciones ACID, seguridad integrada)	Alta (JPA/Hibernate, gestión transaccional robusta)	Media (menor adopción en sistemas clínicos críticos)

Justificación de los criterios de evaluación

A continuación, se presentan los fundamentos técnicos que sustentan la valoración asignada a cada criterio de la comparación.

Eficiencia y rendimiento La evaluación de este criterio se basa en estudios de rendimiento que analizan el comportamiento de APIs bajo escenarios de carga controlados. En este contexto, Godinho et al. (2024) reportan diferencias significativas en el consumo de memoria entre .NET y Java, donde la implementación en Java presenta aproximadamente el doble de uso de RAM en estado de reposo (*idle*), con valores de 1.40 GiB frente a 694 MiB en .NET [18].

En términos de rendimiento, los resultados de throughput indican que ASP.NET Core obtiene un mayor número de respuestas por segundo en escenarios de alta concurrencia. De forma consistente, los benchmarks de TechEmpower [3] confirman que .NET se mantiene entre las tecnologías con mejor desempeño en pruebas de múltiples consultas por solicitud, como se evidencia en la Figura 1.8 [19].

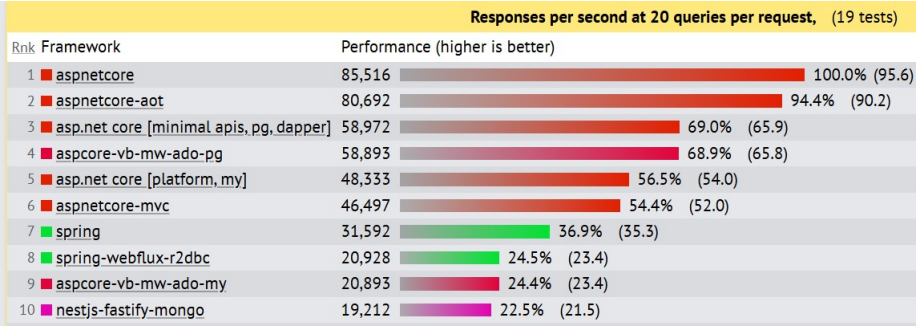

Figura 1.8: Throughput (respuestas por segundo) en .NET, Spring Boot y Node.js (adaptado de TechEmpower Framework Benchmarks)

Asimismo, los resultados de latencia evidencian que .NET presenta tiempos de respuesta más estables bajo incremento de carga.

En escenarios de estrés (*POST*), la Figura 1.9 muestra que el consumo de memoria se mantiene consistente con pruebas previas, observándose que Java requiere aproximadamente el doble de RAM que .NET, mientras que el uso de CPU presenta diferencias mínimas entre ambos frameworks [18].

Escalabilidad A partir de la comparación, se observa que tanto .NET como Spring Boot presentan un comportamiento escalable robusto en entornos distribuidos, siendo

[3] TechEmpower Framework Benchmarks es un conjunto de pruebas estandarizadas que evalúa el rendimiento de frameworks web bajo condiciones homogéneas de carga, permitiendo comparar métricas como throughput y latencia entre diferentes tecnologías como .NET, Java y Node.js.

Figura 1.9: Prueba de estrés sobre operaciones POST: uso de memoria y CPU (adaptado de *Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot* [18])

ampliamente utilizados en arquitecturas de microservicios.

Por su parte, NestJS ofrece una alta capacidad de concurrencia en escenarios ligeros gracias a su modelo de ejecución asíncrono. Sin embargo, los resultados de benchmarks indican que .NET y Spring Boot mantienen un comportamiento más consistente bajo cargas sostenidas [19].

Mantenibilidad De acuerdo con la Tabla correspondiente, .NET y Spring Boot presentan un nivel de mantenibilidad alto, principalmente debido a la consolidación de patrones arquitectónicos como IoC y la existencia de ecosistemas maduros que favorecen la estandarización del desarrollo y facilita las pruebas unitarias.

Por otro lado, NestJS presenta una mantenibilidad media, influenciada por la mayor variabilidad de su ecosistema y la dependencia de múltiples librerías externas en evolución constante.

Adicionalmente, la curva de aprendizaje también influye en la mantenibilidad del sistema, ya que tecnologías con menor complejidad inicial facilitan la incorporación de nuevos desarrolladores y reducen el tiempo de adaptación al proyecto. Esta diferencia puede observarse en la Tabla 1.3.

Madurez tecnológica .NET y Spring Boot presentan una madurez de grado empresarial consolidada, respaldada por su amplia adopción en sistemas productivos y su evolución sostenida a lo largo del tiempo, incluyendo soporte de largo plazo (LTS) en sus versiones actuales.

.NET refuerza su estabilidad mediante su ciclo de lanzamientos LTS y su fuerte integración en entornos corporativos, mientras que Spring Boot se mantiene como el estándar de facto en el ecosistema Java para aplicaciones empresariales distribuidas.

Tabla 1.3: Comparación de la curva de aprendizaje entre tecnologías backend

Tecnología	**Dificultad**	**Factores determinantes**
ASP.NET Core (.NET)	Media-Alta	Herramientas integradas (Visual Studio), arquitectura homogénea y tipado fuerte.
Spring Boot	Alta	Complejidad del ecosistema Java, uso de anotaciones y autoconfiguración.
NestJS	Media-Baja	Sintaxis accesible en TypeScript; complejidad en asincronía y concurrencia.

Por su parte, NestJS, aunque moderno y en crecimiento, presenta una madurez menor al estar basado en el ecosistema JavaScript/Node.js, caracterizado por ciclos de cambio más rápidos y menor estabilidad histórica en sistemas de misión crítica.

Adecuación al dominio clínico En sistemas de salud, la integridad, seguridad e interoperabilidad de los datos constituyen requisitos fundamentales debido a la naturaleza crítica de la información clínica.

En este contexto, .NET proporciona soporte para APIs nativas de protección de datos y estándares como *SMART on FHIR*[4], facilitando la integración con sistemas clínicos externos y el cumplimiento de normativas como HIPAA[5] [20, 21].

Asimismo, su integración con Entity Framework Core permite garantizar propiedades transaccionales de tipo ACID, asegurando consistencia, integridad y trazabilidad en operaciones críticas como el registro y auditoría de eventos adversos.

En el contexto del Instituto Finlay de Vacunas, esta madurez se evidencia en la adopción de plataformas como OpenClinica para la gestión de datos clínicos y el uso de herramientas de inteligencia de negocio como bambooBSC, ambas basadas en el ecosistema Java [22].

Spring Boot presenta una alta adecuación al dominio clínico debido a su robustez transaccional y su amplio uso en sistemas empresariales del sector salud, aunque su capa de persistencia basada en JPA/Hibernate puede incrementar la complejidad en consultas avanzadas.

[4] *SMART on FHIR* (Substitutable Medical Applications and Reusable Technologies on Fast Healthcare Interoperability Resources) es un estándar que permite el desarrollo de aplicaciones interoperables en salud mediante acceso seguro a datos clínicos estructurados sobre FHIR.

[5] HIPAA (Health Insurance Portability and Accountability Act) establece requisitos para la protección y confidencialidad de la información médica.

Por su parte, NestJS presenta una adecuación moderada, debido a su menor adopción en sistemas clínicos críticos y a la dependencia de bibliotecas externas para garantizar funcionalidades de seguridad y cumplimiento normativo.

Selección de la plataforma para el sistema de farmacovigilancia

A partir del análisis realizado, se determina que **.NET** constituye la opción más adecuada para el sistema de farmacovigilancia por las siguientes razones:

1. **Eficiencia de recursos:** Presenta un menor consumo de memoria y un alto rendimiento bajo carga en comparación con otras alternativas, lo que favorece su despliegue en entornos con recursos limitados [18].

2. **Fiabilidad en operación:** Mantiene un comportamiento estable bajo cargas sostenidas, garantizando la integridad de las transacciones críticas en el registro de eventos adversos [18].

3. **Seguridad y cumplimiento:** Dispone de mecanismos nativos para la protección de datos sensibles y cumplimiento de normativas del sector salud, lo que resulta esencial en sistemas clínicos [20].

4. **Mantenibilidad y cohesión del ecosistema:** Su arquitectura integrada, junto con herramientas unificadas y soporte de largo plazo (LTS), facilita la evolución controlada del sistema.

5. **Adecuación al contexto de desarrollo:** La experiencia previa con el ecosistema .NET reduce la curva de aprendizaje y el riesgo de implementación, favoreciendo un desarrollo más eficiente y sostenible en el tiempo.

1.2.4. Tecnologías para el desarrollo del frontend

En el desarrollo de aplicaciones web modernas, la selección de la tecnología de frontend constituye un elemento clave para garantizar la correcta implementación de interfaces de usuario eficientes, escalables y coherentes con las necesidades del sistema.

En este contexto, no se busca identificar exhaustivamente todas las alternativas existentes, sino seleccionar un conjunto representativo de tecnologías que permitan realizar un análisis comparativo basado en criterios técnicos relevantes para aplicaciones clínicas.

En este sentido, se seleccionan React, Angular y Vue por representar tres enfoques distintos en el desarrollo frontend moderno.

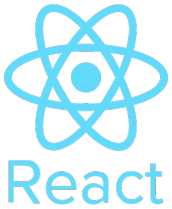

Figura 1.10: Logo de React

React

React es una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces basadas en componentes reutilizables. Su modelo declarativo utiliza un Virtual DOM para optimizar la actualización de la interfaz mediante un proceso de reconciliación [23].

Este mecanismo reduce las operaciones directas sobre el DOM real, mejorando el rendimiento en interfaces con alta frecuencia de cambios, tal como se ilustra en la Figura 1.11.

Sin embargo, React requiere la integración de librerías externas para funcionalidades como enrutamiento y gestión de estado, lo que incrementa la complejidad arquitectónica del frontend.

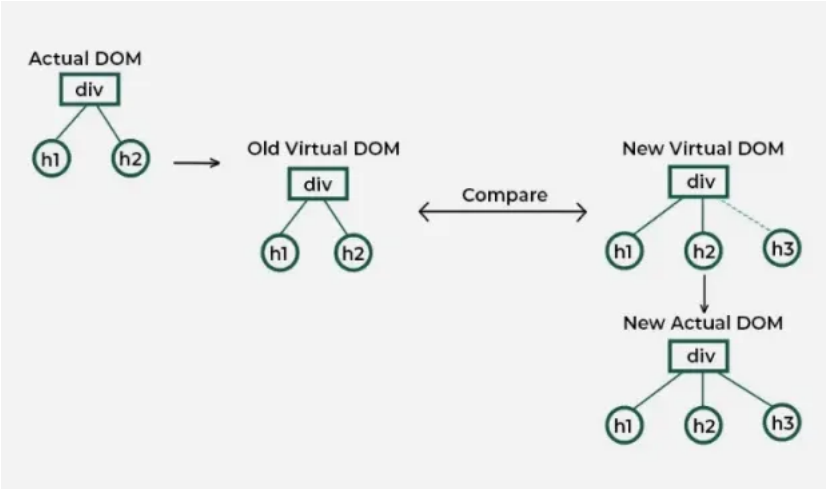

Figura 1.11: Funcionamiento del Virtual DOM en React (adaptado de *Virtual DOM in React JS*, GeeksforGeeks [24])

Figura 1.12: Logo de Angular

Angular

Angular es un framework de desarrollo frontend basado en TypeScript, mantenido por Google, orientado a la construcción de aplicaciones web de gran escala mediante una arquitectura completamente integrada [25].

Su principal característica distintiva es el uso de *two-way data binding*, el cual permite la sincronización automática entre el modelo de datos y la vista, reduciendo la necesidad de manipulación manual de la interfaz. Asimismo, su enfoque "opinionated" define una estructura de desarrollo estandarizada desde el inicio del proyecto, lo que favorece la consistencia arquitectónica en aplicaciones de gran escala.

Además, Angular proporciona una solución integral que incluye enrutamiento, inyección de dependencias, manejo de formularios y herramientas de estructuración de aplicaciones.

Vue

Figura 1.13: Logo de Vue.js

Vue.js es un framework progresivo de JavaScript diseñado para la construcción de interfaces de usuario interactivas. Su diseño incremental permite su adopción gradual, integrándose en proyectos existentes o utilizándose como framework completo [26].

Su principal característica es un sistema de reactividad que sincroniza de forma eficiente los cambios del estado con la interfaz de usuario, facilitando el desarrollo de interfaces declarativas.

Vue combina principios presentes en frameworks como React y Angular, situándose como una alternativa intermedia en términos de flexibilidad y estructura.

1.2.5. Conclusiones sobre los frameworks de frontend expuestos

Aunque los criterios definidos son aplicables de forma general al sistema, su interpretación varía según la capa tecnológica analizada.

En el frontend, la eficiencia se relaciona con el rendimiento en la actualización de la interfaz, la escalabilidad con la gestión de interfaces complejas basadas en componentes, y la adecuación al dominio clínico con la capacidad de soportar la captura, validación y visualización estructurada de datos.

Tabla 1.4: Comparación de tecnologías frontend según criterios de evaluación

Criterio	**React**	**Angular**	**Vue**
Renderizado	Alto (Virtual DOM, reconciliación eficiente)	Medio (change detection basado en zonas)	Alto (reactividad basada en dependencias)
Escalabilidad (UI)	Alta (componentes reutilizables)	Muy alta (arquitectura modular integrada)	Alta (reactividad y composición flexible)
Peso y Rendimiento	Ligero (enfocado solo en la carga de vista, agilizando el tiempo de carga inicial)	Pesado (framework pesado que incluye múltiples librerías integradas)	Muy Ligero (20-30KB ideal para cargas rápidas en redes limitadas)
Mantenibilidad	Alta (componentes + ecosistema flexible)	Muy alta (estructura integral y tipada)	Alta (estructura simple y progresiva)
Curvas de Aprendizaje	Media (ecosistema flexible con integración de librerías externas y gestión del estado)	Alta (framework integral con múltiples conceptos)	Baja-Media (enfoque progresivo y sintaxis accesible)
Madurez	Muy alta (amplia adopción y comunidad)	Muy alta (uso empresarial consolidado)	Alta (ecosistema en crecimiento)
Adecuación clínica (UI)	Alta (gestión de estado y formularios dinámicos)	Muy alta (formularios y validación integrados)	Alta (reactividad y simplicidad en formularios)

A partir de la Tabla 1.4, se identifican diferencias en los enfoques de construcción de interfaces. React y Vue priorizan mecanismos reactivos de actualización eficiente,

mientras que Angular ofrece una solución más estructurada e integrada.

Considerando los criterios evaluados, React presenta un equilibrio entre eficiencia, flexibilidad y mantenibilidad, lo cual resulta adecuado para aplicaciones basadas en componentes dinámicos.

Dado que el sistema propuesto requiere interfaces altamente interactivas orientadas a formularios clínicos, se selecciona React como tecnología base para el desarrollo del frontend.

1.2.6. Sistemas de gestión de bases de datos

En sistemas de farmacovigilancia, la gestión de datos clínicos es un elemento clave debido a la necesidad de almacenar y consultar información sensible asociada a pacientes, eventos adversos y tratamientos.

Por ello, la elección del sistema de gestión de bases de datos impacta directamente en la consistencia, integridad y escalabilidad del sistema.

En el estado del arte se consideran dos enfoques principales: bases de datos relacionales y bases de datos NoSQL, seleccionados por su amplia adopción en sistemas empresariales y por representar paradigmas complementarios de almacenamiento estructurado y flexible.

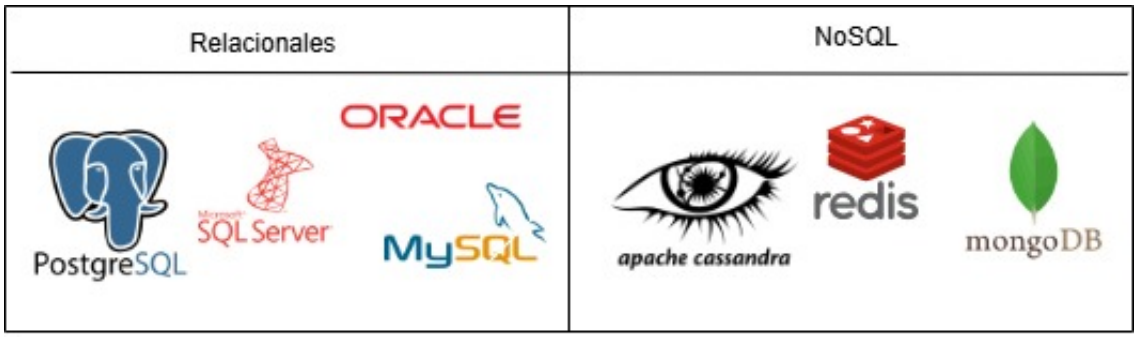

Figura 1.14: Ejemplos de sistemas de gestión de bases de datos relacionales y NoSQL

Bases de datos relacionales

Las bases de datos relacionales se basan en un modelo tabular donde la información se organiza en filas y columnas, estableciendo relaciones mediante claves primarias y foráneas, lo que permite garantizar propiedades como la integridad referencial, la consistencia de los datos y el cumplimiento de transacciones ACID [27].

No obstante, los sistemas relacionales presentan limitaciones cuando se requiere alta flexibilidad en la estructura de los datos o escalabilidad horizontal intensiva, debido a la rigidez del esquema predefinido.

Bases de datos NoSQL

Las bases de datos NoSQL surgen como alternativa a los modelos relacionales, orientadas a proporcionar mayor flexibilidad en la estructura de los datos y mejor adaptación a entornos distribuidos y de gran escala. A diferencia del modelo relacional, no requieren un esquema rígido, lo que permite gestionar datos semi-estructurados (como documentos JSON con campos variables) o no estructurados (como imágenes médicas, registros libres de texto o archivos clínicos) [28].

Sin embargo, esta flexibilidad puede comprometer la consistencia de los datos y la gestión de relaciones complejas, lo que limita su idoneidad en sistemas donde la integridad de la información es crítica, como en el dominio clínico.

1.2.7. Conclusiones sobre los sistemas de gestión de bases de datos expuestos

Se comparan dos paradigmas de bases de datos: relacionales y NoSQL, en función de criterios técnicos relevantes para sistemas clínicos.

Tabla 1.5: Comparación de paradigmas de bases de datos según criterios de evaluación

Criterio	**Relacional (SQL)**	**NoSQL**
Estabilidad	Muy alta (madurez consolidada, transacciones ACID robustas)	Alta (estable, pero dependiente del motor y configuración distribuida)
Mantenibilidad	Alta (esquema estructurado y consistente)	Media (flexibilidad del esquema puede aumentar complejidad)
Adecuación clínica	Muy alta (integridad, trazabilidad y consistencia de datos)	Media (dependiente del diseño de la aplicación)
Curva de aprendizaje	Media (modelado relacional y SQL)	Media-Baja (modelos flexibles y menos estructurados)
Escalabilidad	Media (principalmente vertical, implica el aumento de recursos en un único servidor)	Muy alta (escalabilidad horizontal nativa, implica la incorporación de múltiples nodos distribuidos)
Eficiencia	Alta en datos estructurados	Alta en grandes volúmenes de datos no estructurados

En el contexto del sistema propuesto, se selecciona MySQL como sistema de gestión de bases de datos debido a su alta consistencia, soporte transaccional y adecua-

ción al dominio clínico.

1.2.8. Síntesis comparativa de las tecnologías analizadas

A partir del análisis realizado en las secciones anteriores, se evidencia que las tecnologías evaluadas presentan características diferenciadas en función del componente del sistema al que están orientadas, así como de los paradigmas de diseño que promueven.

En el ámbito del backend, se identifican soluciones orientadas a la construcción de sistemas robustos y mantenibles, destacándose plataformas consolidadas como .NET y Spring Boot por su madurez, soporte arquitectónico y enfoque en aplicaciones empresariales, así como alternativas más recientes como NestJS, que introducen modelos modulares y flexibles dentro del ecosistema JavaScript.

En cuanto al desarrollo de interfaces de usuario, se observa una tendencia hacia arquitecturas basadas en componentes y modelos declarativos, donde tecnologías como React, Angular y Vue ofrecen distintos niveles de estructuración, flexibilidad y control sobre la complejidad de la aplicación, en función de los requerimientos de interacción y dinamismo de la interfaz.

Por otra parte, en el contexto de la persistencia de datos, las tecnologías analizadas reflejan enfoques diferenciados en cuanto al modelado y gestión de la información, lo cual influye directamente en aspectos como la consistencia, la integridad y la adaptabilidad del sistema ante cambios en los datos.

De manera integral, se evidencia que la selección tecnológica no debe realizarse de forma aislada por componente, sino considerando la coherencia entre las distintas capas del sistema. En particular, la combinación de tecnologías backend, frontend y de persistencia debe garantizar un equilibrio entre rendimiento, mantenibilidad y confiabilidad, aspectos especialmente críticos en sistemas orientados a la gestión de información clínica.

Estos elementos constituyen la base para la selección de las tecnologías que serán empleadas en la implementación del sistema, la cual se aborda en el capítulo siguiente.

1.3. Tecnologías y enfoques para la gestión de datos clínicos

La gestión de datos clínicos en sistemas de farmacovigilancia constituye un problema de naturaleza compleja, en el cual convergen requisitos de interoperabilidad semántica, consistencia estructural, trazabilidad de la información y calidad en la captura de los datos. A diferencia de los sistemas de información convencionales, los entornos clínicos demandan un tratamiento riguroso de la información debido a su

impacto directo en procesos de análisis sanitario, toma de decisiones en salud pública y evaluación de la seguridad de las vacunas.

En este contexto, los enfoques tecnológicos asociados a la gestión de datos clínicos no se limitan únicamente a herramientas de almacenamiento o procesamiento, sino que abarcan también estándares de representación de la información, modelos de captura de datos y mecanismos de validación que permiten garantizar la integridad de los registros desde su origen hasta su utilización en procesos analíticos.

Asimismo, la heterogeneidad de las fuentes de datos y la variabilidad en la forma en que los usuarios reportan eventos adversos introduce el denominado problema de "ruido informativo", el cual dificulta la integración y el análisis automatizado de la información clínica. Por esta razón, resulta necesario el uso de estándares y enfoques que permitan normalizar la representación de los datos, así como establecer reglas de validación que aseguren su coherencia y fiabilidad.

De igual forma, los sistemas de farmacovigilancia modernos han evolucionado hacia modelos que combinan distintos paradigmas de captura de información, incluyendo mecanismos de reporte en línea y modelos de captura diferida mediante formularios estructurados, con el objetivo de garantizar la disponibilidad del sistema en escenarios con conectividad limitada sin comprometer la calidad de los datos recolectados.

En las subsecciones siguientes se analizan los principales estándares internacionales utilizados en la representación de información clínica, los modelos de captura de datos empleados en sistemas de autorreporte de eventos adversos, así como los enfoques de validación de datos clínicos, con el objetivo de fundamentar las decisiones de diseño adoptadas en la presente investigación.

1.3.1. Estándares de interoperabilidad clínica

La interoperabilidad en sistemas de información en salud constituye un requisito esencial para garantizar que los datos clínicos puedan ser compartidos, interpretados y reutilizados de manera consistente entre diferentes sistemas y actores del ecosistema sanitario. En el contexto de la farmacovigilancia de vacunas, esta capacidad resulta fundamental para asegurar que la información reportada mantenga su significado original y pueda ser utilizada de forma fiable en procesos de análisis y toma de decisiones.

Uno de los estándares más relevantes en este ámbito es HL7 FHIR (Fast Healthcare Interoperability Resources), desarrollado por la organización HL7 [29], el cual define un modelo basado en recursos que permite estructurar la información clínica de manera modular e interoperable. Este estándar facilita la representación de entidades como pacientes, vacunas administradas y eventos clínicos asociados, permitiendo la integración entre sistemas heterogéneos.

El modelo basado en FHIR introduce una estructura flexible donde cada recur-

so representa una unidad de información clínica que puede ser combinada con otras para describir eventos complejos. En el caso de los eventos adversos asociados a vacunas, esta estructura permite relacionar el paciente, la inmunización recibida y las observaciones clínicas derivadas.

Por otra parte, la estandarización terminológica constituye un elemento clave para evitar ambigüedades en la interpretación de los datos. En este sentido, el diccionario MedDRA (Medical Dictionary for Regulatory Activities) proporciona una terminología controlada para la codificación de eventos adversos [30], permitiendo la representación uniforme de síntomas y reacciones asociadas a vacunas.

Como se observa en la Figura 1.15, MedDRA organiza los términos clínicos en una estructura jerárquica de cinco niveles, que permite representar un mismo concepto clínico con distintos grados de especificidad. Esta jerarquía va desde categorías generales (System Organ Class) hasta términos altamente específicos (Lowest Level Term), lo cual facilita la normalización semántica de los eventos reportados y reduce la ambigüedad en la interpretación de los datos clínicos.

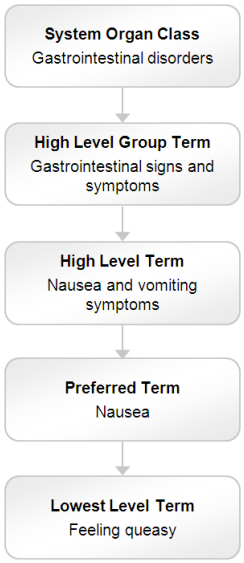

Figura 1.15: Estructura jerárquica del sistema de codificación MedDRA.

De manera complementaria, la Clasificación Internacional de Enfermedades (CIE), desarrollada por la Organización Mundial de la Salud [31], proporciona un marco estandarizado para la codificación de condiciones de salud, lo cual permite enriquecer el contexto clínico de los reportes cuando es necesario.

En conjunto, la adopción de estándares de interoperabilidad y terminologías controladas como FHIR, MedDRA y CIE permite mejorar la calidad, consistencia y utilidad de los datos clínicos, constituyendo un elemento fundamental en el diseño de

sistemas orientados al reporte estructurado de eventos adversos asociados a vacunas.

1.3.2. Modelos de captura de datos clínicos

La captura de datos clínicos constituye una etapa crítica en el diseño de sistemas de información en salud, ya que determina en gran medida la calidad, consistencia y utilidad de la información registrada. En el contexto de los sistemas de reporte de eventos adversos asociados a vacunas, una captura inadecuada puede derivar en pérdidas de información relevante, ambigüedades en los datos o dificultades en su posterior procesamiento.

En la práctica, pueden identificarse diferentes modelos de captura de datos clínicos en función del nivel de estructuración de la información desde su origen.

El modelo de captura estructurada se basa en la utilización de formularios con campos previamente definidos, lo cual permite restringir los valores de entrada y facilitar la validación automática de los datos. Este enfoque es ampliamente utilizado en sistemas de información en salud modernos, debido a su capacidad para mejorar la calidad del dato desde su origen y facilitar su procesamiento posterior.

El modelo semi-estructurado constituye un enfoque intermedio, donde la información se organiza bajo una plantilla definida pero permite cierto grado de flexibilidad en su completitud. Un ejemplo representativo de este enfoque es el formulario en formato PDF utilizado en sistemas de farmacovigilancia como el Vaccine Adverse Event Reporting System (VAERS), el cual permite la recolección de información incluso en escenarios donde no existe conectividad continua o acceso a plataformas en línea [3].

Finalmente, el modelo no estructurado se basa en la captura de información en forma de texto libre o descripciones narrativas. Aunque este enfoque puede aportar mayor riqueza descriptiva, presenta limitaciones significativas en términos de procesamiento automático, normalización de datos e integración con sistemas de análisis clínico.

Desde la perspectiva del diseño de sistemas de información, los modelos estructurados y semi-estructurados resultan más adecuados para escenarios de farmacovigilancia, ya que permiten garantizar un mayor control sobre la calidad de los datos desde su origen, facilitando su integración con procesos posteriores de validación, normalización y análisis.

En consecuencia, la selección del modelo de captura debe equilibrar la precisión de la información, la facilidad de uso para el usuario final y la capacidad de integración con los procesos de análisis del sistema.

1.3.3. Herramientas para la validación reactiva de formularios

La validación de datos en tiempo de captura constituye un mecanismo esencial para garantizar la calidad y consistencia de la información en sistemas de autorreporte clínico. En este contexto, la validación reactiva se refiere al proceso de verificación de los datos introducidos por el usuario de manera inmediata, es decir, antes de su envío al servidor, permitiendo detectar y corregir errores en etapas tempranas del flujo de interacción.

Este tipo de validación resulta especialmente relevante en sistemas de reporte de eventos adversos asociados a vacunas, donde la precisión de los datos ingresados impacta directamente en la confiabilidad del análisis posterior.

Las técnicas de validación pueden clasificarse en validaciones sintácticas, orientadas a verificar el formato de los datos (como fechas, números o identificadores), y validaciones semánticas básicas, enfocadas en garantizar coherencia lógica entre los campos del formulario. Ejemplos de estas incluyen la verificación de campos obligatorios, rangos válidos de valores o consistencia entre fechas relacionadas.

En el desarrollo de aplicaciones modernas, existen múltiples herramientas que facilitan la implementación de validación reactiva. En el ecosistema frontend basado en React, bibliotecas como React Hook Form y Formik permiten gestionar formularios complejos con validación en tiempo real [32] [33]. De manera similar, en Angular se emplean formularios reactivos que integran validación mediante el módulo Reactive Forms [34].

En el ámbito del backend, frameworks como .NET incorporan bibliotecas como FluentValidation, que permiten definir reglas de validación de forma declarativa y reutilizable [35]. En ecosistemas basados en Node.js, bibliotecas como Joi o Zod proporcionan mecanismos de validación estructurada de datos antes de su persistencia.

La utilización de estas herramientas permite reducir significativamente la probabilidad de inserción de datos inconsistentes, mejorando la calidad de la información desde su punto de origen y complementando los procesos posteriores de normalización e integración en el sistema.

1.3.4. Calidad e integración de datos clínicos

La calidad de los datos clínicos constituye un factor determinante en la utilidad de los sistemas de información en salud, especialmente en contextos donde la información es utilizada para el análisis de eventos adversos asociados a vacunas. En este tipo de sistemas, la presencia de datos incompletos, inconsistentes o ambiguos puede comprometer significativamente la fiabilidad de los resultados obtenidos.

Desde una perspectiva de ingeniería de software, la calidad de los datos puede

evaluarse a partir de múltiples dimensiones, entre las que se incluyen la integridad, la consistencia, la exactitud, la completitud y la trazabilidad. Estas propiedades garantizan que la información almacenada represente de manera fiel los eventos del mundo real y pueda ser utilizada de forma confiable en procesos posteriores de análisis.

La integración de datos clínicos se refiere al proceso mediante el cual la información capturada y validada en el sistema es unificada bajo un modelo común, permitiendo su interoperabilidad y su posterior procesamiento. En este sentido, la utilización de un modelo de datos estructurado resulta esencial para evitar la fragmentación de la información y facilitar su análisis global.

En el contexto de los sistemas de autorreporte de eventos adversos, la integración de datos se apoya en el uso de estándares de interoperabilidad como HL7 FHIR [29], que permite la representación estructurada de entidades clínicas, y en terminologías controladas como MedDRA [30] y la Clasificación Internacional de Enfermedades (CIE) de la Organización Mundial de la Salud [31], que aseguran la normalización semántica de los conceptos médicos.

Estos estándares permiten establecer un lenguaje común entre sistemas heterogéneos, facilitando la interoperabilidad, la coherencia semántica de los datos y su reutilización en diferentes contextos clínicos.

Asimismo, la integración adecuada de los datos clínicos permite establecer relaciones coherentes entre las distintas entidades del sistema, como el paciente, la vacunación administrada y las reacciones adversas reportadas, garantizando la trazabilidad completa del evento clínico.

En conjunto, la aplicación de principios de calidad de datos y mecanismos de integración estructurada permite mejorar la confiabilidad del sistema, reducir la ambigüedad en la interpretación de la información y facilitar su uso en procesos de análisis clínico y toma de decisiones.

1.4. Conclusiones del capítulo

A partir del análisis realizado en el presente capítulo, se evidencia que los sistemas de autorreporte de eventos adversos asociados a vacunas constituyen herramientas fundamentales dentro de los procesos de farmacovigilancia moderna. Sin embargo, los sistemas estudiados como VAERS, NotificaRAM y V-safe presentan limitaciones relacionadas con la heterogeneidad en la captura de datos, la dependencia de estructuras de reporte específicas y, en algunos casos, restricciones en la validación temprana de la información reportada.

En cuanto a las tecnologías utilizadas para la implementación de sistemas de información en este dominio, se observa la existencia de múltiples alternativas consolidadas para el desarrollo del backend, frontend y la gestión de bases de datos. No obstante,

la selección de dichas tecnologías debe responder a criterios de robustez, escalabilidad, mantenibilidad y, especialmente, a la capacidad de garantizar la integridad de los datos en contextos clínicos.

Asimismo, el análisis de los modelos de captura de datos clínicos evidencia que los enfoques estructurados y semi-estructurados constituyen las alternativas más adecuadas para sistemas de autorreporte, debido a su capacidad para reducir la ambigüedad en la información y facilitar su posterior procesamiento. En particular, los modelos basados en formularios estructurados y formatos interoperables como los utilizados en sistemas de referencia permiten mejorar la calidad del dato desde su origen.

Por otra parte, la incorporación de herramientas de validación reactiva en el punto de captura representa un elemento esencial para la reducción de errores en la entrada de datos, contribuyendo directamente a la consistencia de la información almacenada. De forma complementaria, los mecanismos de integración y calidad de datos clínicos, apoyados en estándares como HL7 FHIR, MedDRA y la Clasificación Internacional de Enfermedades (CIE), permiten establecer un marco común para la representación y normalización de la información clínica.

En conjunto, el estado del arte analizado permite identificar que la principal problemática en los sistemas actuales no radica únicamente en la disponibilidad de tecnologías, sino en la integración coherente de modelos de captura, validación, estandarización e interoperabilidad de datos clínicos. Esta situación fundamenta la necesidad de proponer una solución que unifique estos elementos dentro de un modelo computacional consistente orientado al reporte estructurado de eventos adversos asociados a vacunas.

Capítulo 2

Propuesta

Capítulo 3

Detalles de Implementación y Experimentos

Conclusiones

Conclusiones

Recomendaciones

Recomendaciones

Apéndice A

Formularios de Referencia

A.1. Formulario VAERS - Octubre 2025

En el Anexo A.1 se incluye el formulario oficial del Vaccine Adverse Event Reporting System (VAERS) correspondiente a octubre de 2025. Este formulario constituye el instrumento estándar utilizado para la notificación de eventos adversos posteriores a la vacunación en los Estados Unidos.

VAERS Vaccine Adverse Event Reporting System
www.vaers.hhs.gov

Adverse events are possible reactions or problems that occur during or after vaccination. Items 2, 3, 4, 5, 6, 17, 18 and 21 are ESSENTIAL and should be completed. Patient identity is kept confidential. Instructions are provided on the last two pages.

INFORMATION ABOUT THE PATIENT WHO RECEIVED THE VACCINE (Use Continuation Page if needed)

1. Patient name: (first) (last)
Street address:
City: State: County:
ZIP code: Phone: ( ) Email:
2. Date of birth: (mm/dd/yyyy)
3. Sex: ☐ Male ☐ Female
4. Date and time of vaccination: (mm/dd/yyyy) Time: hh:mm ☐AM ☐PM
5. Date and time adverse event started: (mm/dd/yyyy) Time: hh:mm ☐AM ☐PM
6. Age at vaccination: Years Months
7. Today's date: (mm/dd/yyyy)
8. Pregnant at time of vaccination?: ☐ Yes ☐ No ☐ Unknown
(If yes, describe the event, any pregnancy complications, and estimated due date if known in item 18)
9. Prescriptions, over-the-counter medications, dietary supplements, or herbal remedies being taken at the time of vaccination:
10. Allergies to medications, food, or other products:
11. Other illnesses at the time of vaccination and up to one month prior:
12. Chronic or long-standing health conditions:

INFORMATION ABOUT THE PERSON COMPLETING THIS FORM

13. Form completed by: (name)
Relation to patient: ☐ Healthcare professional/staff ☐ Patient (yourself) ☐ Parent/guardian/caregiver ☐ Other:
Street address: ☐ Check if same as item 1
City: State: ZIP code:
Phone: ( ) Email:
14. Best doctor/healthcare professional to contact about the adverse event: Name: Phone: ( ) Ext:

INFORMATION ABOUT THE FACILITY WHERE VACCINE WAS GIVEN

15. Facility/clinic name:
Fax: ( )
Street address: ☐ Check if same as item 13
City:
State: ZIP code:
Phone: ( )
16. Type of facility: (Check one)
☐ Doctor's office, urgent care, or hospital
☐ Pharmacy or store
☐ Workplace clinic
☐ Public health clinic
☐ Nursing home or senior living facility
☐ School or student health clinic
☐ Home
☐ Other: ☐ Unknown

WHICH VACCINES WERE GIVEN? WHAT HAPPENED TO THE PATIENT?

17. Enter all vaccines given on the date listed in item 4: (Route is HOW vaccine was given, Body site is WHERE vaccine was given) Use Continuation Page if needed

Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series
select			select	select	select
select			select	select	select
select			select	select	select
select			select	select	select

18. Describe the adverse event(s), treatment, and outcome(s), if any: (symptoms, signs, time course, etc.)
Use Continuation Page if needed
19. Medical tests and laboratory results related to the adverse event(s): (include dates)
Use Continuation Page if needed
20. Has the patient recovered from the adverse event(s)?: ☐ Yes ☐ No ☐ Unknown
21. Result or outcome of adverse event(s): (Check all that apply)
☐ Doctor or other healthcare professional office/clinic visit
☐ Emergency room/department or urgent care
☐ Hospitalization: Number of days (if known)
Hospital name:
City: State:
☐ Prolongation of existing hospitalization (vaccine received during existing hospitalization)
☐ Life threatening illness (immediate risk of death from the event)
☐ Disability or permanent damage
☐ Patient died – Date of death: (mm/dd/yyyy)
☐ Congenital anomaly or birth defect
☐ None of the above

ADDITIONAL INFORMATION

22. Any other vaccines received within one month prior to the date listed in item 4: Use Continuation Page if needed

Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series	Date Given
select			select	select	select	
select			select	select	select	

23. Has the patient ever had an adverse event following any previous vaccine?: (If yes, describe adverse event, patient age at vaccination, vaccination dates, vaccine type, and brand name)
☐ Yes ☐ No ☐ Unknown
24. Patient's race: (Check all that apply) ☐ American Indian or Alaska Native ☐ Asian ☐ Black or African American ☐ Native Hawaiian or Other Pacific Islander ☐ White ☐ Unknown ☐ Other:
25. Patient's ethnicity: ☐ Hispanic or Latino ☐ Not Hispanic or Latino ☐ Unknown
26. Immuniz. proj. report number: (Health Dept use only)

COMPLETE ONLY FOR U.S. MILITARY/DEPARTMENT OF DEFENSE (DoD) RELATED REPORTS

27. Status at vaccination: ☐ Active duty ☐ Reserve ☐ National Guard ☐ Beneficiary ☐ Other:
28. Vaccinated at Military/DoD site: ☐ Yes ☐ No

FORM FDA VAERS 2.0 (10/25) SAVE

Figura A.1: Formulario VAERS - Octubre 2025

Glosario

Glosario

desidentificados Proceso mediante el cual se eliminan o transforman datos que permiten identificar a una persona de forma directa o indirecta (nombres, números de identificación, datos de ubicación, fechas) para proteger la privacidad de los individuos. 5

Lista de acrónimos

VAERS — Vaccine Adverse Event Reporting System

V-safe — Vaccine Safety Assessment System (CDC)

FDA — Food and Drug Administration

CDC — Centers for Disease Control and Prevention

AEMPS — Agencia Española de Medicamentos y Productos Sanitarios

SEFV-H — Sistema Español de Farmacovigilancia de Medicamentos de Uso Humano

FEDRA — Farmacovigilancia Española, Datos de Reacciones Adversas

LOPDGDD — Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales

Bibliografía

[1] U.S. Food and Drug Administration. *VAERS Overview*. Accedido: 06 de abril de 2026. 2024. URL: https://www.fda.gov/vaccines-blood-biologics/vaccine-adverse-events/vaers-overview (vid. pág. 2).

[2] Centers for Disease Control and Prevention. *About the Vaccine Adverse Event Reporting System (VAERS)*. Accedido: 06 de abril de 2026. 2024. URL: https://www.cdc.gov/vaccine-safety-systems/vaers/index.html (vid. pág. 2).

[3] Vaccine Adverse Event Reporting System. *Vaccine Adverse Event Reporting System (VAERS)*. Accedido: 06 de abril de 2026. 2024. URL: https://vaers.hhs.gov/ (vid. págs. 2, 23).

[4] Agencia Española de Medicamentos y Productos Sanitarios. *Manual de uso NotificaRAM*. Accedido: 06 de abril de 2026. 2025. URL: https://www.notificaram.es/ (vid. pág. 3).

[5] Agencia Española de Medicamentos y Productos Sanitarios. *Sistema Español de Farmacovigilancia de Medicamentos de Uso Humano*. Accedido: 06 de abril de 2026. 2025. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-de-medicamentos-de-uso-humano/identificacion-de-riesgos/sistema-espanol-de-farmacovigilancia-de-medicamentos-de-uso-humano/ (vid. pág. 3).

[6] Agencia Española de Medicamentos y Productos Sanitarios. *Información sobre el acceso a los datos de FEDRA*. Accedido: 06 de abril de 2026. 2026. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-de-medicamentos-de-uso-humano/informacion-sobre-el-acceso-a-los-datos-de-fedra/ (vid. págs. 3, 5).

[7] Agencia Española de Medicamentos y Productos Sanitarios. *Notificación de sospechas de reacciones adversas a medicamentos (RAM) de uso humano*. Accedido: 06 de abril de 2026. 2026. URL: https://www.aemps.gob.es/medicamentos-de-uso-humano/farmacovigilancia-de-medicamentos-de-uso-humano/notificacion-de-sospechas-de-reacciones-adversas-a-medicamentos-ram-de-uso-humano/ (vid. pág. 3).

[8] Centers for Disease Control and Prevention. *V-safe After Vaccination Health Checker*. Accedido: 06 de abril de 2026. 2024. URL: https://vsafe.cdc.gov (vid. pág. 3).

[9] Centers for Disease Control and Prevention. *About V-safe*. Accedido: 06 de abril de 2026. 2025. URL: https://www.cdc.gov/vaccine-safety-systems/v-safe/index.html (vid. págs. 3, 4).

[10] Tanya R. Myers et al. «The v-safe after vaccination health checker: Active vaccine safety monitoring during CDC's COVID-19 pandemic response». En: *Vaccine* 41.7 (2023). Disponible en PubMed Central, págs. 1310-1318. DOI: 10.1016/j.vaccine.2022.12.031 (vid. págs. 3, 4).

[11] Microsoft. *.NET Documentation*. https://dotnet.microsoft.com/en-us/. Accessed: 2026-04-10. 2024 (vid. pág. 8).

[12] Microsoft Developer Division. *Architect Modern Web Applications with ASP.NET Core and Azure*. Accessed: 2026-04-10. Microsoft, 2023. URL: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/ (vid. pág. 8).

[13] Kirk Larkin, Steve Smith y Brandon Dahler. *Dependency injection in ASP.NET Core*. 2026. URL: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection (vid. pág. 8).

[14] Wikipedia contributors. *Entity Framework Core*. 2026. URL: https://en.wikipedia.org/wiki/Entity_Framework (vid. pág. 8).

[15] VMware, Inc. *Spring Boot Documentation*. https://spring.io/projects/spring-boot. Accessed: 2026-04-10. 2024 (vid. pág. 9).

[16] Craig Walls. *Spring in Action*. 5th. Manning Publications, 2019 (vid. pág. 9).

[17] NestJS Team. *NestJS Documentation*. https://docs.nestjs.com/. Accessed: 2026-04-10. 2024 (vid. pág. 9).

[18] Antonio Godinho et al. «Performance Comparison of RESTful Web APIs using a Test Suite: .NET vs. Java Spring Boot». En: *Journal of Software & Systems Development* (2024). DOI: 10.5171/2024.478010 (vid. págs. 11, 12, 14).

[19] TechEmpower. *Web Framework Benchmarks*. https://www.techempower.com/benchmarks/. Accessed: April 20, 2026. 2025 (vid. págs. 11, 12).

[20] Rushil Bhuptani. *Building HIPAA-Compliant Healthcare Apps with .NET: Best Practices and Pitfalls*. Publicado originalmente en febrero de 2025 y modificado el 10 de marzo de 2026. Mar. de 2026. URL: https://www.avidclan.com/blog/building-hipaa-compliant-healthcare-apps-with-dot-net-best-practices-and-pitfalls/ (vid. págs. 13, 14).

[21] Darena Health. *Build a SMART on FHIR App in ASP.NET Core (C#) from Scratch with MeldRx*. Video tutorial sobre la implementación de estándares HL7 FHIR en el ecosistema .NET. YouTube. 2024. URL: https://youtu.be/UKTisHLeqn4?si=wy_h-HnsJKoBFYya (vid. pág. 13).

[22] Ing. Judith de las Mercedes Carbonell Alfonso. «Propuesta de diseño del Cuadro de Mando Integral para la toma de decisiones en el Instituto Finlay de Vacunas». Tutor: Dr. Luis del Castillo Sánchez. Tesis en opción al grado de Máster en Gestión de Información. La Habana, Cuba: Universidad de La Habana, Facultad de Economía, 2016 (vid. pág. 13).

[23] Meta Open Source. *React Documentation*. https://react.dev. Accessed: 2026-04-11. 2024 (vid. pág. 15).

[24] GeeksforGeeks. *ReactJS Virtual DOM*. Accedido en abril de 2026. 2026. URL: https://www.geeksforgeeks.org/reactjs/reactjs-virtual-dom/ (vid. pág. 15).

[25] Google. *Angular Documentation*. https://angular.io. Accessed: 2026-04-11. 2024 (vid. pág. 16).

[26] Vue.js Team. *Vue.js Guide*. https://vuejs.org. Accessed: 2026-04-11. 2024 (vid. pág. 16).

[27] Ramez Elmasri y Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 2016 (vid. pág. 18).

[28] ABM Moniruzzaman y Syed Akhter Hossain. «Nosql database: New era of databases for big data analytics-classification, characteristics and comparison». En: *arXiv preprint arXiv:1307.0191* (2013) (vid. pág. 19).

[29] HL7 International. *FHIR Overview*. Accedido: 2026-04-11. 2024. URL: https://www.hl7.org/fhir/overview.html (vid. págs. 21, 25).

[30] International Council for Harmonisation (ICH). *Medical Dictionary for Regulatory Activities (MedDRA)*. Accedido: 2026-04-11. 2024. URL: https://www.meddra.org/ (vid. págs. 22, 25).

[31] World Health Organization. *International Classification of Diseases (ICD)*. Accedido: 2026-04-11. 2024. URL: https://www.who.int/standards/classifications/classification-of-diseases (vid. págs. 22, 25).

[32] *React Hook Form Documentation*. https://react-hook-form.com/. Accedido: 2026-04-11 (vid. pág. 24).

[33] *Formik Documentation*. https://formik.org/. Accedido: 2026-04-11 (vid. pág. 24).

[34] *Angular Reactive Forms Guide*. https://angular.io/guide/reactive-forms. Accedido: 2026-04-11 (vid. pág. 24).

[35] *FluentValidation Documentation*. https://docs.fluentvalidation.net/. Accedido: 2026-04-11 (vid. pág. 24).